Dynamic Prototype Spawning and Pruning with Hybrid Kalman-Momentum Updates in Continuous Hopfield Networks

1 Executive Summary

Static continuous Hopfield prototype memories impose a rigid, uniform capacity hyperparameter across all target categories. However, real-world data distributions exhibit pronounced heterogeneity: simple, compact classes require only a few prototypes to achieve geometric coverage, while multi-modal or visually complex classes suffer from severe under-representation when restricted to a fixed prototype budget.

This proposal introduces an adaptive memory architecture with **Dynamic Prototype Spawning and Pruning** driven by assignment support and uncertainty propagation.

Building on the online hybrid Kalman-momentum framework of Dynamic Prototype Learning, the memory layer dynamically self-regulates its capacity:

1. **Masked Pre-Allocated Fixed-Size Buffers:**

   Pre-allocates memory buffers up to a maximum capacity ceiling $K_{\max}$ per class with a binary activity mask $m \in \{0,1\}^N$, eliminating dynamic tensor reallocation in PyTorch while supporting variable active prototype counts.

2. **Pruning via Data Starvation ($|\mathcal{S}_{j,n}|$):**

   Prototypes that consistently fail to attract minimum training assignment support τ_{support} over T_{prune} consecutive epochs are deactivated, concentrating softmax attention on representative vectors while preserving a minimum invariant of $K_{\min} = 1$ active prototype per category.

3. **Spawning via Uncertainty Fission ($P_{j,n}$):**

   When a prototype exhibits high uncertainty $P_{j,n} > \tau_{\text{uncert}}$ driven by intra-cluster dispersion and classification error, it undergoes fission into two distinct child prototypes separated along the first principal component of its assigned training cluster, with velocity cleared ($V = 0$) and uncertainty reset to P_0.

4. **Masked Centered-Cosine Retrieval:**

   Attention weights apply an infinite negative penalty to inactive buffer slots, computing associative readouts as convex combinations of raw active prototypes without modifying the downstream classifier.

2 Problem Statement and Theoretical Motivation

Let

$$f_\theta : \mathcal{X} \to \mathbb{R}^D$$

map an input $x \in \mathcal{X}$ to an embedding $z \in \mathbb{R}^D$, and let

$$g_\phi : \mathbb{R}^D \to \mathbb{R}^C$$

denote the linear classification head.

In the previous experiment, each class is allocated an identical, static capacity of K prototypes:

$$N = C \cdot K.$$

This static design exhibits three primary shortcomings:

1. **The Uniformity Flaw (Capacity Mismatch):**

   Different classes possess fundamentally different geometric topologies in $\mathbb{R}^D$. Allocating identical prototype budgets forces simple unimodal classes to maintain redundant prototypes, while complex multi-modal classes suffer from prototype starvation.

2. **Dead Prototype Attention Dilution:**

   In associative Hopfield retrieval, inactive, poorly positioned, or redundant prototypes continue to compete for softmax probability mass. This dilutes retrieval attention and degrades noise-filtering capacity under severe corruption.

3. **Under-Representation of High-Variance Sub-Clusters:**

   When a single prototype attempts to cover two distinct visual sub-modes, its position is pulled into the low-density space between the modes, resulting in poor associative recall for both.

The experiment resolves these limitations by treating prototype capacity as a dynamic, data-driven property governed by local cluster uncertainty and empirical assignment support.

3 Network Architecture and Masked Memory State

3.1 Pre-Allocated Masked Buffer Layout

To preserve static graph execution, enable full GPU/CPU compilation, and avoid memory reallocation during training:

- Total buffer capacity is fixed at

  $$N = C \cdot K_{\max},$$

  where $K_{\max}$ represents the upper bound on allowable prototypes per category.

- Initial online spontaneous admission is bounded by an initial capacity K_{init}, where

$$1 \leq K_{\text{init}} \leq K_{\max}.$$

  Additional slots between K_{init} and $K_{\max}$ can only be unlocked via structural spawning.

3.2 State Variables per Buffer Slot

Each slot $j \in \{1, \ldots, N\}$ maintains:

- **Prototype Vector:**

$$M_j \in \mathbb{R}^D,$$

  storing raw embedding coordinates.

- **Class Label:**

$$c_j \in \{0, \ldots, C-1\},$$

  binding the slot to a specific category.

- **Velocity Vector:**

$$V_j \in \mathbb{R}^D,$$

  recording momentum displacement.

- **Uncertainty Estimate:**

$$P_j \in \mathbb{R}^+,$$

  initialized to P_0.

- **Binary Activity Mask:**

$$m_j \in \{0, 1\},$$

  indicating whether the slot participates in retrieval and assignment.

- **Starvation Streak Counter:**

$$\text{streak}_j \in \mathbb{N}_0,$$

  counting consecutive epochs with inadequate assignment support.

- **Recent Assignment Support:**

$$\text{support}_j \in \mathbb{N}_0,$$

  recording $|\mathcal{S}_{j,n}|$.

A slot is active if and only if $m_j = 1$ and its entry identifier is valid.

3.3 Coordinate Centering

A frozen global mean vector $\mu \in \mathbb{R}^D$ is extracted from all clean training embeddings at the end of the warm-up epoch (E_{warmup}) and held constant throughout all subsequent epochs:

$$\mu = \frac{1}{|\mathcal{D}_{\text{train}}|} \sum_{x_i \in \mathcal{D}_{\text{train}}} f_\theta(x_i).$$

4 Dual-Phase Training and Dynamic Allocation Lifecycle

Training alternates between **intra-epoch online minibatch processing** and **epoch-end structural adaptation**.

4.1 Training Pipeline Overview

The complete training pipeline consists of two phases:

1. **Phase 1: Intra-Epoch Online Minibatch Stream**

   For each minibatch:

   1. Forward pass through the direct CNN:

      $$z_i = f_\theta(x_i), \qquad \hat{y}_{\text{direct}} = \arg\max g_\phi(z_i).$$

   2. Perform masked Hopfield retrieval using active slots ($m_j = 1$).
   3. Check accuracy-guided eligibility: Direct correct and Hopfield incorrect.
   4. Find the nearest active same-class prototype:

      $$j^* = \arg\min_j d(z_i, M_j).$$

   5. Compare the distance against the merge threshold:
      - If $d^* \leq \tau_{\text{merge}}$, apply online Kalman-momentum merging.
      - If $d^* > \tau_{\text{merge}}$ and the class count is below K_{init}, add a new slot.
   6. Perform direct loss backward propagation and optimizer update of (θ, ϕ) using cross-entropy.

2. **Phase 2: Epoch-End Structural Adaptation (`adapt_prototypes`)**

   1. Assign all training embeddings of each class to their nearest active prototype to partition $\mathcal{S}_{j,n}$.
   2. Compute support and update starvation streaks.

3. Propagate local uncertainty using cluster dispersion σ_j and classification error.
4. Prune prototypes whose starvation streak exceeds T_{prune}, while preserving at least $K_{\min}$ active prototypes per class.
5. Spawn new prototypes through uncertainty-driven principal component fission when the uncertainty threshold is exceeded and capacity is available.

5 Intra-Epoch Online Minibatch Processing

During the epoch, minibatches of size B_{train} update both the direct CNN parameters (θ, ϕ) and the active prototype states.

5.1 Masked Centered-Cosine Retrieval

Given query embedding $q \in \mathbb{R}^D$, centered directional keys are formed for both the query and active memory prototypes:

$$\tilde{q} = \frac{q - \mu}{\|q - \mu\|_2 + \epsilon_{\text{norm}}}, \qquad \tilde{M}_j = \frac{M_j - \mu}{\|M_j - \mu\|_2 + \epsilon_{\text{norm}}}.$$

Masked attention weights $w \in \Delta^{N-1}$ enforce an infinite penalty on inactive slots. In implementation, inactive logits should be masked to $-\infty$ before applying softmax:

$$\ell_j = \begin{cases} \beta \tilde{q}^\top \tilde{M}_j, & m_j = 1, \\ -\infty, & m_j = 0. \end{cases}$$

The attention weights are then computed as:

$$w_j = \frac{\exp(\ell_j)}{\sum_{k=1}^{N} \exp(\ell_k)}.$$

The associative output h linearly combines the raw prototypes:

$$h = \sum_{j=1}^{N} w_j M_j.$$

5.2 Admission Eligibility and Candidate Planning

For sample (x_i, y_i), direct prediction is

$$\hat{y}_i^{\text{direct}} = \arg\max g_\phi(z_i)$$

and memory prediction is

$$\hat{y}_i^{\text{hopfield}} = \arg\max g_\phi(h_i).$$

Under accuracy-guided admission, z_i is eligible for memory mutation if and only if:

$$\text{Eligible}(i) \iff \hat{y}_i^{\text{direct}} = y_i \quad \wedge \quad \hat{y}_i^{\text{hopfield}} \neq y_i.$$

Eligible embeddings search among active prototypes sharing category y_i:

$$j^* = \arg \min_{\substack{j \\ m_j=1,\, c_j=y_i}} \left[1 - \tilde{z}_i^\top \tilde{M}_j\right],$$

$$d^* = d(z_i, M_{j^*}).$$

Merge ($d^* \leq \tau_{\text{merge}}$). Prototype M_{j^*} updates its position via the hybrid Kalman-momentum rule:

$$K_{j^*} = \frac{P_{j^*}}{P_{j^*} + (1 - A_{\text{train}}) + \epsilon_{\text{stab}}}.$$

$$V_{j^*} \leftarrow \gamma V_{j^*} + (1-\gamma)(z_i - M_{j^*}).$$

$$M_{j^*} \leftarrow M_{j^*} + K_{j^*} V_{j^*}.$$

$$P_{j^*} \leftarrow (1 - K_{j^*}) P_{j^*} + Q.$$

Add ($d^* > \tau_{\text{merge}}$ and class count below K_{init}). A previously inactive slot is activated:

$$m_{\text{new}} \leftarrow 1,$$

$$M_{\text{new}} \leftarrow z_i, \qquad V_{\text{new}} \leftarrow 0, \qquad P_{\text{new}} \leftarrow P_0.$$

Skip. If the class active count has reached K_{init}, additions are suppressed during online streaming.

6 Epoch-End Structural Adaptation (`adapt_prototypes`)

At the conclusion of each post-warmup training epoch, the full clean training set is passed through feature extractor f_θ to evaluate global geometric cluster structures.

6.1 Global Cluster Assignment and Support Computation

For each category $c \in \{0, \ldots, C-1\}$, let

$$\mathcal{D}_c = \{z_i \mid y_i = c\}$$

denote all training embeddings of class c.

Each embedding $z_i \in \mathcal{D}_c$ is assigned to its nearest active same-class prototype under centered cosine distance:

$$\mathcal{S}_{j,n} = \left\{ z_i \in \mathcal{D}_c \;\middle|\; j = \arg \min_{\substack{k \\ m_k=1,\, c_k=c}} d(z_i, M_k) \right\}.$$

The assignment support $|\mathcal{S}_{j,n}|$ quantifies empirical data density around prototype M_j.

6.2 Cluster Uncertainty and Local Dispersion Propagation

For each active prototype with non-empty assignment ($|\mathcal{S}_{j,n}| > 0$), the following quantities are computed.

1. Local Classification Accuracy

$$\mathrm{Acc}_{j,n} = \frac{1}{|\mathcal{S}_{j,n}|} \sum_{z_i \in \mathcal{S}_{j,n}} \mathbb{I}(g_\phi(z_i) = c).$$

2. Intra-Cluster Geometric Dispersion

$$\sigma_{j,n} = \mathrm{StdDev}_{z_i \in \mathcal{S}_{j,n}} \left(d(z_i, M_j) \right).$$

3. Local Observation Noise

$$R_{j,n} = (1 - \mathrm{Acc}_{j,n}) + \lambda_{\mathrm{disp}} \sigma_{j,n} + \epsilon_{\mathrm{stab}}.$$

where λ_{disp} scales the influence of spatial dispersion. High intra-cluster spread or low local accuracy increases observation noise.

4. Local Uncertainty Update

$$K_{j,n} = \frac{P_{j,n}}{P_{j,n} + R_{j,n}}.$$

$$P_{j,n+1} = (1 - K_{j,n}) P_{j,n} + Q + \sigma_{j,n}.$$

Injecting local dispersion $\sigma_{j,n}$ into the process noise guarantees that prototypes attempting to cover multi-modal or wide sub-clusters accumulate elevated uncertainty over time.

6.3 Pruning via Assignment Support Starvation

Prototypes that attract fewer than τ_{support} samples fail to capture significant data density. If

$$|\mathcal{S}_{j,n}| < \tau_{\text{support}},$$

the starvation streak increments:

$$\text{streak}_{j,n} = \text{streak}_{j,n-1} + 1.$$

Otherwise, the starvation streak resets:

$$\text{streak}_{j,n} = 0.$$

Pruning Rule Prototype j is marked for deactivation if:

$$\text{streak}_{j,n} \geq T_{\text{prune}}.$$

Class Invariant To guarantee valid Hopfield retrieval at all times, pruning is constrained such that each class preserves at least $K_{\min} = 1$ active prototype:

$$|\{k \mid m_k = 1,\ c_k = c\}| > K_{\min}.$$

When pruned, slot j is deactivated:

$$m_j \leftarrow 0, \qquad c_j \leftarrow -1,$$

$$V_j \leftarrow 0, \qquad P_j \leftarrow P_0, \qquad \text{streak}_j \leftarrow 0.$$

6.4 Spawning via Uncertainty Fission (Principal Component Splitting)

A prototype with persistently elevated uncertainty $P_{j,n+1}$ indicates that a single vector cannot adequately represent the variance of its assigned sub-cluster $\mathcal{S}_{j,n}$.

Spawning Rule For surviving active prototypes, fission is triggered when:

$$P_{j,n+1} > \tau_{\text{uncert}}$$

and

$$|\{k \mid m_k = 1,\ c_k = c\}| < K_{\max},$$

with

$$|\mathcal{S}_{j,n}| \geq 2.$$

Upon triggering fission, the following steps are performed.

1. Principal Direction Extraction Center the assigned embeddings:

$$\bar{z} = \frac{1}{|\mathcal{S}_{j,n}|} \sum_{z_i \in \mathcal{S}_{j,n}} z_i.$$

Compute the singular value decomposition on centered embeddings:

$$\bar{\mathcal{S}}_{j,n} = U \Sigma V^\top.$$

The leading right singular vector v_1 (first row of $V^\top$) identifies the axis of maximum intra-cluster variance, with principal standard deviation

$$\sigma_1 = \frac{\Sigma_{1,1}}{\sqrt{|\mathcal{S}_{j,n}| - 1}}.$$

2. Symmetric Perturbation Vector

$$\delta_j = \alpha_{\text{split}} \cdot \sigma_1 \cdot v_1.$$

where α_{split} is the splitting scale factor.

3. Child Prototype Instantiation Child (a): Retained Slot j

$$M_j^{(a)} = M_j + \delta_j,$$

$$V_j \leftarrow 0, \qquad P_j \leftarrow P_0, \qquad \text{streak}_j \leftarrow 0.$$

Child (b): Allocated Slot j_{free}

Find an inactive slot j_{free} where

$$m_{j_{\text{free}}} = 0.$$

Initialize the new child:

$$M_{j_{\text{free}}}^{(b)} = M_j - \delta_j,$$

$$c_{j_{\text{free}}} \leftarrow c, \qquad m_{j_{\text{free}}} \leftarrow 1,$$

$$V_{j_{\text{free}}} \leftarrow 0, \qquad P_{j_{\text{free}}} \leftarrow P_0.$$

Both child prototypes begin with clean momentum history and baseline uncertainty, allowing them to specialize into distinct sub-clusters during subsequent training epochs.

7 Comparative Method Ablations

The experiment evaluates four update rules sharing the identical CNN backbone, dynamic spawning/pruning engine, and masked retrieval:

1. **EMA Baseline:**

   Applies static exponential moving average during online merges (α_{ema}) while executing support pruning and uncertainty spawning at epoch boundaries.

2. **Hybrid Kalman-Momentum (Full Proposed Method):**

   Combines online momentum trajectory smoothing (V_j) and Kalman gain gating (K_j) with epoch-end dynamic spawning and pruning.

3. **Momentum-Only Ablation:**

   Maintains momentum smoothing but sets Kalman trust to unity ($K_j \equiv 1.0$) during online merges.

4. **Kalman-Only Ablation:**

   Applies Kalman gating without momentum history ($V_j \equiv z_i - M_j$) during online merges.

8 Evaluation Protocol and Diagnostics

8.1 Accuracy Metrics

Evaluated across independent seeds S on disjoint clean validation and test splits:

- **Direct CNN Baseline:**

  Evaluated with $h = z$ directly into g_ϕ.

- **Dynamic Hopfield Memory:**

  Evaluated with masked associative retrieval h into g_ϕ.

8.2 Comprehensive Noise Robustness Benchmark

Both the Direct CNN baseline and all four memory update rules are evaluated under identical noise draws across:

- **Additive Gaussian Noise:**
  Standard deviation grid:
  $$\Sigma = \{\sigma_1, \ldots, \sigma_G\}.$$
- **Impulsive Salt-and-Pepper Noise:**
  Corruption probability grid:
  $$\mathcal{P} = \{p_1, \ldots, p_P\}.$$
- **Spatial Square Occlusion:**
  Cutout size grid:
  $$\mathcal{S} = \{s_1, \ldots, s_O\}.$$

Direct comparison tables report baseline versus memory accuracy at every individual noise level.

8.3 Dynamic Memory Allocation Diagnostics

At every epoch boundary, the following structural metrics are logged:

- **Active Prototype Count:**
  $$|\text{active}| = \sum_{j=1}^{N} m_j.$$
- **Per-Class Prototype Distribution:**
  Active prototype counts for each category $c \in \{0, \ldots, C-1\}$.
- **Cumulative Spawning Count:**
  Total fission events executed.
- **Cumulative Pruning Count:**
  Total deactivations executed.
- **Structural Churn Rate:**
  Cumulative (Spawns + Prunes) per seed, measuring geometric stability.

9 Summary of Hyperparameters

Table 1: Summary of hyperparameters governing dynamic prototype allocation, pruning, spawning, and associative retrieval.

Symbol	Description	Mathematical Role
C	Number of target categories	Categorical class partitioning
D	Feature embedding dimension	Vector space dimensionality
K_{init}	Initial online prototype capacity	Online spontaneous admission ceiling per class
$K_{\max}$	Maximum prototype buffer ceiling	Pre-allocated buffer capacity per class ($N = C \cdot K_{\max}$)
$K_{\min}$	Minimum prototype retention invariant	Lower bound on active prototypes per class ($K_{\min} = 1$)
E_{total}	Total training epochs	Training duration
E_{warmup}	Warm-up epochs	Feature stabilization before memory initialization
B_{train}	Training minibatch size	Minibatch chunk size
B_{eval}	Evaluation minibatch size	Inference evaluation batch size
η	Optimization learning rate	Adam step size for (θ, ϕ)
τ_{merge}	Merge cosine distance threshold	Boundary separating merge mutation versus slot addition
τ_{support}	Minimum assignment support threshold	Minimum data count $\lvert\mathcal{S}_{j,n}\rvert$ to avoid starvation
T_{prune}	Starvation patience	Consecutive low-support epochs required for pruning
τ_{uncert}	Spawning uncertainty threshold	Trigger threshold for prototype fission
α_{split}	Principal component split scale	Perturbation magnitude along principal axis v_1
λ_{disp}	Dispersion penalty coefficient	Weight of spatial dispersion in observation noise R_j
α_{ema}	Static moving average rate	EMA baseline merge weighting
γ	Momentum discount coefficient	Velocity exponential decay rate in $[0, 1)$
Q	Base process noise	Uncertainty injection rate per step

Continued on next page

Table 1 – Continued from previous page

Symbol	**Description**	**Mathematical Role**
P_0	Initial prototype uncertainty	Prior variance assigned to new and spawned prototypes
ϵ_{stab}	Numerical stability epsilon	Divisor guard for Kalman observation noise
ϵ_{norm}	Vector normalization epsilon	Divisor guard for unit vector normalization
β	Hopfield inverse temperature	Softmax scaling factor for associative retrieval